NAME: AAKASH S ROLL NO: 231901001

EX. NO.4

QUERYING LEDGER DATA

AIM

To query and retrieve ledger data from a Hyperledger Fabric blockchain network using JavaScript chaincode, and to verify asset information stored in the world state database.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

PROCEDURE

Step 1: Set Up the Fabric Test Network

Navigate to the Hyperledger Fabric test

network. cd ~/fabric-dev/fabric-

samples/test-network Stop any

previously running Fabric network.

./network.sh down

Start the Fabric network, create the application channel, and enable Certificate Authority. ./network.sh up createChannel -ca

Step 2: Deploy the JavaScript Chaincode

Deploy the Asset Transfer Basic JavaScript chaincode.

./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript

```
vboxuser@Ubuntu:~$ cd ~/fabric-dev/fabric-samples/test-network
./network.sh down
./network.sh up createChannel -ca
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/ch
aincode-javascript
source setOrg1.sh
Using docker and docker compose
Stopping network
[+] down 10/10
 ✔ Container ca_orderer                  Removed                              0.2s
 ✔ Container ca_org1                     Removed                              0.3s
 ✔ Container orderer.example.com         Removed                              0.3s
 ✔ Container peer0.org1.example.com      Removed                              0.3s
 ✔ Container ca_org2                     Removed                              0.3s
 ✔ Container peer0.org2.example.com      Removed                              0.3s
 ✔ Network fabric_test                   Removed                              0.3s
 ✔ Volume compose_orderer.example.com    Removed                              0.1s
 ✔ Volume compose_peer0.org1.example.com Removed                              0.1s
 ✔ Volume compose_peer0.org2.example.com Removed                              0.1s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
```

Step 3: Set the Organization Environment

Load the Org1 environment.

source setOrg1.sh

Set the TLS certificate path for Org1.

export ORG1_TLS=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1. example.com/tls/ca.crt

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke
-o localhost:7050 --ordererTLSHostnameOverride orderer.example.com \
  --tls --cafile $ORDERER_CA -C mychannel -n basic \
  -c '{"function":"InitLedger","Args":[]}'
2026-08-30 15:53:40.857 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> C
haincode invoke successful. result: status:200
```

Set the TLS certificate path for Org2.

export
ORG2_TLS=${PWD}/organizations/peerOrganizations/org2.example.com/peers
/peer0.org2. example.com/tls/ca.crt

```
[]
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export ORG1_TLS=${PWD}
/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/t
ls/ca.crt
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ export ORG2_TLS=${PWD}
/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/t
ls/ca.crt
```

Step 4: Initialize the Ledger

Invoke the InitLedger function to initialize the ledger with

sample assets. peer chaincode invoke -o localhost:7050 --

ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --
peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses
localhost:9051 --tlsRootCertFiles $ORG2_TLS -c
'{"function":"InitLedger","Args":[]}'

Wait for the command to complete successfully before continuing.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke
-o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafil
e $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCert
Files $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '
{"function":"InitLedger","Args":[]}'
2026-08-30 15:57:57.738 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> C
haincode invoke successful. result: status:200
```

Step 5: Query All Assets

Retrieve all assets stored in the world state.

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -
C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"d
ocType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad
","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3
","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"
yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValu
e":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asse
t"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":
15,"docType":"asset"}]
```

Step 6: Query a Specific Asset

Retrieve the details of asset1.

peer chaincode query -C mychannel -n basic -c
'{"Args":["ReadAsset","asset11"]}'

Step 7: Create a New Asset

Create a new asset named asset20.

peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCertFiles $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}'

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -
C mychannel -n basic -c '{"Args":["ReadAsset","asset1"]}'
{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"do
cType":"asset"}
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke
-o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafil
e $ORDERER_CA -C mychannel -n basic --peerAddresses localhost:7051 --tlsRootCert
Files $ORG1_TLS --peerAddresses localhost:9051 --tlsRootCertFiles $ORG2_TLS -c '
{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}'
2026-08-30 15:58:51.231 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> C
haincode invoke successful. result: status:200 payload:"{\"ID\":\"asset20\",\"Co
lor\":\"Green\",\"Size\":8,\"Owner\":\"Arun\",\"AppraisedValue\":1500}"
```

Step 8: Query the Newly Created Asset

Retrieve the details of asset20.

peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset20"]}'

Step 9: Query All Assets Again

Retrieve all assets again to verify that the newly created asset has been

added to the ledger. peer chaincode query -C mychannel -n basic -c

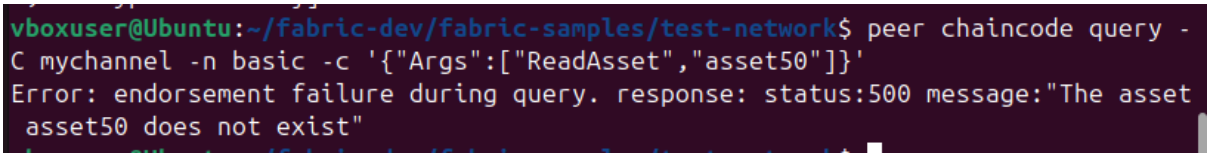
'{"Args":["GetAllAssets"]}'

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -
C mychannel -n basic -c '{"Args":["ReadAsset","asset20"]}'
{"AppraisedValue":1500,"Color":"Green","ID":"asset20","Owner":"Arun","Size":8}
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -
C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"d
ocType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad
","Size":5,"docType":"asset"},{"AppraisedValue":1500,"Color":"Green","ID":"asset
20","Owner":"Arun","Size":8},{"AppraisedValue":500,"Color":"green","ID":"asset3"
,"Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"y
ellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue
":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset
"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":1
5,"docType":"asset"}]
```

Step 10: Query a Non-Existing Asset

Attempt to retrieve an asset that does not exist in the ledger.

peer chaincode query -C mychannel -n basic -'{"Args":["ReadAsset","asset50"]}'



Step 11: Stop the Fabric Network

Stop and remove the Fabric test network after completing the experiment.

./network.sh down

FLOW OF LEDGER QUERY PROCESS

Start Fabric Network

|

▼

Deploy JavaScript Chaincode

|

▼

Set Organization Environment

|

▼

Initialize

Ledger

|

▼

Query All Assets

|

▼

Query Specific Asset

|

▼

Create New Asset

|

▼

Query New Asset

|

▼

Verify Updated Ledger

|

▼

Query Non-Existing Asset

|

▼

Stop Fabric Network

RESULT

The ledger data stored in the Hyperledger Fabric blockchain network was successfully queried using JavaScript chaincode. The initialized assets were retrieved from the world state, a specific asset was verified, a new asset was created and queried, and the updated ledger was verified successfully.